

Karlova Univerzita
Přírodovědecká fakulta
Katedra aplikované geoinformatiky a kartografie



Algoritmy počítačové kartografie

Geometrické vyhledávání bodu

Aliaksandra Sparysh
Jolana Václavková

Praha 2026

Kapitola 1

Zadání

Úloha č. 1: Geometrické vyhledávání bodu

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, \dots, P_n\}$, analyzovaný bod q .

Výstup: $P_i, q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q .

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

Kapitola 2

Údaje o bonusových úlohách

Krok	Hodnocení
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
<i>Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.</i>	+10b
<i>Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.</i>	+5b
<i>Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.</i>	+5b
<i>Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.</i>	+2b
<i>Zvýraznění všech polygonů pro oba výše uvedené singulární případy.</i>	+3b
<i>Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).</i>	+10b
<i>Řešení pro polygony s dírami.</i>	+10b
<i>Načtení vstupních dat ze *.shp</i>	+10b
<i>Zpracování textové zprávy v LaTeXu</i>	+5b
Max celkem:	70b

Kapitola 3

Popis a rozbor problému

Point location problem

Často řešenou úlohou v oblasti počítačové grafiky, digitální kartografie, či geografických informačních systémů (GIS) je určování vzájemného topologického vztahu mezi zadaným referenčním bodem a uzavřenou oblastí. Tento úkol, v odborné literatuře standardně označovaný jako Point-in-Polygon Problem (PIPP), poskytuje exaktní informaci o tom, zda se zkoumaný bod nachází v interiéru (uvnitř), exteriéru (vně), případně přímo na hranici souvislého geometrického útvaru (polygonu).

Náročnost algoritmického aparátu potřebného pro řešení této úlohy se diametrálně liší v závislosti na geometrických vlastnostech analyzovaného útvaru. Zatímco zjištění polohy bodu vůči striktně konvexním polygonům lze provést pomocí jednodušších a výpočetně méně náročných metod (např. testováním polohy bodu vůči polorovinám), v praxi se převážně pracuje s polygony nekonvexními. Tyto složitější útvary vyžadují robustnější analytické přístupy, které dokáží zohlednit specifické rozložení vrcholů a případnou existenci děr.

Předložená práce se zabývá řešením problému PIPP nad komplexní polygonovou mapou. Geografická data jsou v tomto případě uložena v tzv. špagetovém datovém modelu. Hlavním specifikem a úskalím tohoto modelu je úplná absence explicitně definovaných topologických vztahů – neexistují zde žádné informace o sousednosti jednotlivých polygonů či sdílení hran. Z hlediska definice problému to znamená, že polohu bodu nelze odvodit z okolí, ale je nutné ji analyzovat vůči každému polygonu v dané množině zcela nezávisle.

Kapitola 4

Popisy algoritmů formálním jazykem

4.1 Ray Crossing algorithm

Metoda Ray Crossing slouží k určení, zda se bod q nachází uvnitř daného polygonu P . Princip metody spočívá ve vedení vodorovné přímky procházející testovaným bodem. Tato přímka je v implementaci chápána jako dvojice opačně orientovaných paprsků, tedy pravého a levého paprsku. Sledují se průsečíky těchto paprsků s hranicí polygonu.

Pravé průsečíky jsou ukládány do čítače k , levé průsečíky do čítače k_l . Pokud je počet pravých průsečíků lichý, bod leží uvnitř polygonu. Pokud je počet pravých průsečíků sudý, bod leží vně polygonu. Současně je porovnávána parita pravých a levých průsečíků, což slouží k ošetření hraničních případů.

$$k(q, P) \bmod 2 = \begin{cases} 1, & q \in P \\ 0, & q \notin P \end{cases}$$

Algoritmus je invariantní vůči volbě směru paprsku r , což znamená, že výsledek by měl být při korektním ošetření speciálních případů stejný bez ohledu na zvolený směr paprsku. V praxi je však nutné věnovat pozornost singulárním situacím. Problematický je zejména případ, kdy paprsek prochází přímo vrcholem polygonu, protože by mohlo dojít k chybnému započítání dvou segmentů najednou. Dalším problémem je situace, kdy je paprsek kolineární s hranou polygonu.

Pro zjednodušení výpočtu byla v algoritmu použita redukce souřadnic vzhledem k testovanému bodu q . Tím je bod q přesunut do počátku lokální souřadnicové soustavy:

$$x'_i = x_i - x_q, \quad y'_i = y_i - y_q.$$

Po této transformaci odpovídá pravý paprsek kladné části osy x' a levý paprsek záporné části osy x' . Pro každou hranu polygonu (p_{i-1}, p_i) se nejprve ověřuje, zda daná hrana protíná vodorovnou přímku vedenou testovaným bodem. Hrana je uvažována pouze tehdy, pokud její koncové body leží v různých polorovinách vzhledem k této přímce:

$$(y'_i > 0 \wedge y'_{i-1} \leq 0) \vee (y'_{i-1} > 0 \wedge y'_i \leq 0).$$

Pokud je podmínka splněna, vypočítá se souřadnice průsečíku hrany s osou x' :

$$x'_m = \frac{x'_i y'_{i-1} - x'_{i-1} y'_i}{y'_i - y'_{i-1}}.$$

Pokud platí

$$x'_m > 0,$$

průsečík leží na pravém paprsku a zvýší se čítač pravých průsečíků:

$$k = k + 1.$$

Pokud platí

$$x'_m < 0,$$

průsečík leží na levém paprsku a zvýší se čítač levých průsečíků:

$$k_l = k_l + 1.$$

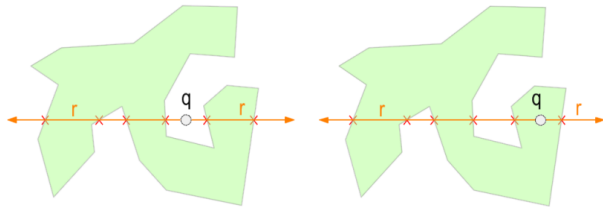
Po průchodu všemi hranami polygonu se nejprve porovná parita pravých a levých průsečíků. Pokud se parity liší, bod je vyhodnocen jako bod ležící na hranici polygonu:

$$(k \bmod 2) \neq (k_l \bmod 2) \Rightarrow q \in \partial P.$$

Pokud jsou parity stejné, rozhoduje parita pravých průsečíků:

$$k \bmod 2 = \begin{cases} 1, & q \in P, \\ 0, & q \notin P. \end{cases}$$

V implementaci jsou před samotným výpočtem navíc samostatně ošetřeny singulární případy, tedy situace, kdy bod leží přímo na hraně polygonu nebo kdy je totožný s některým z jeho vrcholů.



Obrázek 4.1: Schéma algoritmu Ray crossing podle umístění bodu q

4.2 Winding Number algorithm

Metoda Winding Number využívá k určení polohy bodu odlišný geometrický princip než Ray Crossing. Základní myšlenka spočívá v tom, že si v testovaném bodě q představíme pozorovatele, který postupně sleduje všechny vrcholy polygonu P . Pokud bod q leží uvnitř polygonu, směr pohledu při obejití celého polygonu opíše plný úhel 2π . Pokud bod leží vně polygonu, jednotlivé změny úhlů se navzájem vyruší.

Výpočet je založen na sumaci orientovaných úhlů mezi dvojicí vektorů vedených z bodu q do sousedních vrcholů polygonu. Celková hodnota Winding Number se zapisuje jako

$$\Omega(q, P) = \frac{1}{2\pi} \sum_{i=1}^n \omega(p_i, q, p_{i+1}),$$

kde $\omega(p_i, q, p_{i+1})$ označuje orientovaný úhel mezi body p_i , q a p_{i+1} . Hodnota Ω tedy vyjadřuje počet oběhů polygonu kolem testovaného bodu.

Celkový orientovaný úhel lze označit jako

$$S(q, P) = \sum_{i=1}^n \omega_i.$$

Normalizovaná hodnota Winding Number je potom

$$\Omega(q, P) = \frac{1}{2\pi} S(q, P).$$

V implementaci se však přímo pracuje se součtem orientovaných úhlů $S(q, P)$. Bod je považován za vnitřní, pokud je absolutní hodnota tohoto součtu přibližně rovna plnému úhlu:

$$||S(q, P)| - 2\pi| < \varepsilon.$$

Rozhodovací pravidlo lze tedy zapsat jako

$$||S(q, P)| - 2\pi| < \varepsilon \Rightarrow q \in P,$$

jinak platí

$$q \notin P.$$

Pro určení znaménka orientovaného úhlu je nutné stanovit polohu bodu q vzhledem k hraně polygonu. K tomu se používá determinantový test orientace:

$$t = x_{i+1} - x_i y_{i+1} - y_i x_q - x_i y_q - y_i.$$

Podle znaménka determinantu se určuje, zda bod leží v levé nebo pravé polorovině vzhledem k orientované hraně:

$$t > 0 \Rightarrow q \in \sigma_l, \quad t < 0 \Rightarrow q \in \sigma_r, \quad t = 0 \Rightarrow q \in p_i p_{i+1}.$$

Pokud bod q leží v levé polorovině, je příslušný úhel uvažován s kladným znaménkem. Pokud leží v pravé polorovině, je úhel uvažován se záporným znaménkem:

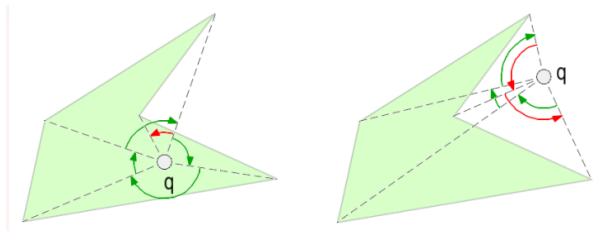
$$\omega_i = \{ +\omega_i, q \in \sigma_l, -\omega_i, q \in \sigma_r.$$

Celková suma orientovaných úhlů tedy závisí na tom, zda polygon bod q skutečně obíhá. Výhodou metody Winding Number je její robustnější geometrická interpretace a lepší chování u některých složitějších konfigurací polygonu. Nevýhodou je vyšší výpočetní náročnost oproti Ray Crossing algoritmu, protože je nutné počítat úhly mezi vektory. Celková časová složitost však zůstává lineární, tedy

$$O(n),$$

kde n je počet vrcholů polygonu.

Stejně jako u algoritmu Ray Crossing jsou i zde před samotným výpočtem samostatně ošetřeny singulární případy. Nejprve se testuje, zda bod q nesplývá s některým vrcholem polygonu. Následně se ověřuje, zda bod neleží na hraně polygonu. Teprve pokud žádná z těchto situací nenastane, je provedena samotná metoda Winding Number.



Obrázek 4.2: Schéma algoritmu Winding number podle umístění bodu q

4.3 Rychlé vyhledání pomocí min–max boxu

Před spuštěním samotného algoritmu Point-in-Polygon je pro každý polygon vypočten jeho obalový obdélník, označovaný jako min–max box. Tento krok slouží jako jednoduchá optimalizace, která umožňuje rychle vyloučit polygony, které testovaný bod určitě nemohou obsahovat. Min–max box je určen minimálními a maximálními hodnotami souřadnic všech vrcholů daného polygonu.

Testovaný bod $q = [x_q, y_q]$ může ležet uvnitř polygonu pouze tehdy, pokud zároveň leží uvnitř jeho obalového obdélníku:

$$x_{\min} \leq x_q \leq x_{\max} \quad \wedge \quad y_{\min} \leq y_q \leq y_{\max}.$$

Pokud tato podmínka není splněna, polygon je z dalšího testování vyřazen. Není tedy nutné nad ním spouštět algoritmus Ray Crossing ani Winding Number. Tento postup snižuje počet testovaných polygonů a urychluje výpočet, zejména u rozsáhlejších polygonových map.

4.4 Vyhodnocení polygonů s dírami

V případě polygonů s dírami nestačí testovat pouze jednu hranici polygonu. Takový polygon je tvořen více prstenci. První prstenec reprezentuje vnější hranici polygonu a další prstence reprezentují vnitřní díry. Bod q náleží polygonu pouze tehdy, pokud leží uvnitř vnější hranice a současně neleží uvnitř žádné vnitřní díry.

Formálně lze tento vztah zapsat jako

$$q \in P \Leftrightarrow q \in R_0 \wedge q \notin R_j, \quad j = 1, 2, \dots, m,$$

kde R_0 označuje vnější prstenec polygonu a R_j označuje jednotlivé vnitřní prstence. Tento princip je použit při výpočtu polohy bodu i při samotném vykreslování polygonů. Při vizualizaci je využito pravidlo Odd-Even Fill, díky kterému zůstávají vnitřní díry polygonu

Kapitola 5

Problematické situace a ošetření těchto situací v kódu

Bod totožný s vrcholem polygonu

Z hlediska geometrie představuje vrchol polygonu 0dimenzionální simplex. Situace, kdy testovaný bod splývá s některým vrcholem, je z problematická především proto, že takový bod může být současně součástí více sousedících polygonů. V klasické variantě algoritmu Ray Crossing by navíc průchod paprsku vrcholem mohl vést k nekorektnímu započtení průsečíku. U algoritmu Winding Number by zase mohlo dojít k degeneraci některého z uvažovaných vektorů, a tím i k numerické nestabilitě při výpočtu úhlu.

Z tohoto důvodu je v implementaci detekce vrcholu provedena ještě před samotným hlavním testem polohy bodu. Pro každý vrchol polygonu je vypočtena eukleidovská vzdálenost mezi testovaným bodem q a daným vrcholem. Pokud je tato vzdálenost menší nebo rovna zvolené toleranci, je bod klasifikován jako bod ležící ve vrcholu. Implementace vrací speciální návratovou hodnotu -2, která umožňuje tento stav dále odlišit od ostatních variant, kde bod může vůči polygonu ležet (univtř, vně, na hraně). Toto řešení je použito jak u algoritmu Ray Crossing, tak u algoritmu Winding Number. Tímto oddělujeme singularitu od samotného rozhodovacího mechanismu algoritmů.

V případě, že je bod vyhodnocen jako totožný s vrcholem, aplikace zvýrazní všechny polygony, které tento vrchol sdílejí. Tato logika je implementována v hlavní třídě aplikace při vyhodno-

```

for pol in poly_rings:
    n = len(pol)
    for i in range(n):
        #Check vertex hit
        dist_v = sqrt((q.x() - pol[i].x())**2 + (q.y() - pol[i].y())**2)
        if dist_v <= self.tolerance:
            return -2

```

Obrázek 5.1: Řešení situace, kdy je bod totožný s vrcholem polygonu

cení návratových stavů algoritmu a předání seznamu polygonů vykreslovací třídě. Uživatel tak nezíská pouze textovou informaci, ale i grafickou interpretaci situace.

Bod ležící na hraně polygonu

Hrana polygonu představuje 1-dimenzionální simplex, a proto je nutné tento případ řešit samostatně. Pokud by bod ležící na hraně nebyl explicitně detekován, mohl by být v závislosti na numerické přesnosti výpočtu a konfiguraci segmentů klasifikován nekonzistentně, například jednou jako bod uvnitř polygonu a jindy jako bod vně polygonu. V implementaci není poloha bodu na hraně určována pouze pomocí vzdálenosti od přímky. Takový postup by nebyl dostatečný, protože bod může ležet na přímce určené hranou, ale současně mimo samotnou úsečku mezi dvěma sousedními vrcholy. Proto je nutné současně ověřit kolinearitu bodu s hranou a také to, zda se bod nachází v rozsahu daného segmentu.

U algoritmu Ray Crossing je tato situace zachycena při výpočtu průsečíku hrany s vodorovnou přímkou vedenou testovaným bodem. Pokud vypočtená souřadnice průsečíku leží v místě testovaného bodu, tedy

$$|x'_m| \leq \varepsilon,$$

je bod vyhodnocen jako bod ležící na hraně polygonu.

U algoritmu Winding Number je bod na hraně detekován pomocí nulového vektorového součinu a úhlu blízkého hodnotě π . Vektorový součin vyjadřuje kolinearitu vektorů vedených z testovaného bodu do sousedních vrcholů hrany. Pokud zároveň platí

$$|\vec{u} \times \vec{v}| < \varepsilon$$

a

$$|\omega - \pi| < \varepsilon,$$

potom testovaný bod leží mezi oběma vrcholy dané hrany, tedy na hranici polygonu. Algoritmus v takovém případě vrací hodnotu -1 .

Stejně jako v případě vrcholů je i zde stav dále zpracován na úrovni třídy MainForm. Pokud je detekováno, že bod leží na hraně, aplikace seskupí všechny polygony, u nichž byl tento stav zjištěn, a graficky je zvýrazní.

```
if len(found_polygons_in) > 1:
    mb.setText(f"Point is INSIDE {len(found_polygons_in)} overlapping polygons")
    self.Canvas.setHighlightedPolygons(found_polygons_in)
elif len(found_polygons_in) == 1:
    mb.setText("Point is INSIDE the polygon")
    self.Canvas.setHighlightedPolygons(found_polygons_in)
elif len(found_polygons_vert) > 0:
    mb.setText(f"Point is exactly on a VERTEX shared by {len(found_polygons_vert)} polygon(s)")
    self.Canvas.setHighlightedPolygons(found_polygons_vert)
elif len(found_polygons_edge) > 0:
    mb.setText(f"Point is on a BOUNDARY shared by {len(found_polygons_edge)} polygon(s)")
    self.Canvas.setHighlightedPolygons(found_polygons_edge)
else:
    mb.setText("Point is OUTSIDE all polygons")
    self.Canvas.setHighlightedPolygons([])
```

Obrázek 5.2: Ukázka kódu pro identifikaci polohy bodu a polygonu

Polygony s dírami

Z hlediska datové reprezentace i interpretace pro nás byly polygony s dírami nejkompexnějším případem celé úlohy. Takový polygon totiž není tvořen pouze jednou uzavřenou hranicí, ale skládá se z více kružnic (rings), přičemž jedna z nich reprezentuje vnější obrys a ostatní vymezují oblasti, které do polygonu nenáleží. Nestačí posuzovat pouze to, zda bod leží uvnitř vnější hranice polygonu, ale je nutné současně ověřit, zda se nenachází uvnitř některé z jeho vnitřních částí. Tento problém jsme řešili již na úrovni načítání vstupních dat. Při zpracování formátu Shapefile jsme jednotlivé části polygonu (tj. jeho prstence) načítali samostatně a ukládali je jako seznam více kružnic. Díky tomu jsme zachovali kompletní topologickou strukturu polygonu a mohli s ní dále pracovat v celém průběhu aplikace.

Takto získanou strukturu jsme následně předávali jak do výpočetních algoritmů, tak do vykreslovací části programu. V algoritmech jsme jednotlivé prstence polygonu procházeli postupně. Zásadní roli hrála také vizualizace. Při vykreslování jsme využili objekt QPainterPath s pravidlem vyplňování Odd-Even Fill, které odpovídá principu sudého a lichého počtu průchodů hranicemi. Díky tomu zůstávají vnitřní kružnice (díry) nevyplněné a polygon je zobrazen korektně.

Stejný princip jsme aplikovali i při zvýraznění výsledného polygonu, čímž jsme zajistili konzistentní chování jak v analytické, tak ve vizuální části aplikace.

```
# We iterate through the individual parts of the polygon (islands, holes, etc.).
record_polygons = []
for p in range(num_parts):
    part_points = []
    start_pt = parts_indices[p]
    end_pt = parts_indices[p+1]

    # Loading points for a specific part (properly indented)
    for i in range(start_pt, end_pt):
        start = points_start_index + (i * 16)
        x, y = struct.unpack("<dd", record_content[start:start+16])
        part_points.append([x, y])

    #Add the loaded points to the list of parts of this polygo
    if len(part_points) > 0:
        record_polygons.append(np.array(part_points))

if record_polygons:
    polygons.append(record_polygons)
```

Obrázek 5.3: Část kódu pro rozklad polygonu na jednotlivé části a extrakci souřadnic

```
for poly_rings in self.__pols:
    path = QPainterPath()
    path.setFillRule(Qt.FillRule.OddEvenFill)
    for ring in poly_rings:
        path.addPolygon(ring)
    qp.drawPath(path)
```

Obrázek 5.4: Vykreslovací smyčka pro geometrické tvary v grafickém okně aplikace

Kapitola 6

Vstupní data

Pro běh a testování implementovaných algoritmů (Ray Crossing a Winding Number) byla zvolena reálná geografická data, konkrétně se jedná o kraje ČR. Tato data obsahují detailní geometrii hranic, což z nich dělá ideální a dostatečně komplexní vstup pro testování problému polohy bodu v nekonvexních oblastech. Hlavní město Praha zde tvoří zastoupení polygonu s dírou. Vstupní data jsou poskytována ve standardním vektorovém formátu ESRI Shapefile (.shp).

Zásadním bodem této části implementace bylo samotné zpracování vstupních souborů. Namísto využití vysokoúrovňových externích knihoven, které by prostorová data načetly automaticky, byl proces čtení formátu Shapefile přímo naprogramován. Tento postup vyžadoval vytvoření vlastního parseru pro čtení binárních dat. Implementovaný algoritmus nejprve načítá hlavní hlavičku souboru pro zjištění metadat a globálního obalového obdélníku (bounding box). Následně sekvenčně prochází jednotlivé záznamy (kraje), u kterých analyzuje jejich typ a iterativně načítá samotné souřadnice lomových bodů (vrcholů), jež tvoří hranice polygonů.

Takto extrahované souřadnice jsou v reálném čase transformovány a ukládány do vnitřních datových struktur aplikace. Tím je zajištěno, že jsou prostorová data z formátu Shapefile úspěšně převedena do podoby, nad kterou mohou následně efektivně operovat naše lokalizační algoritmy.

Kapitola 7

Výstupní data

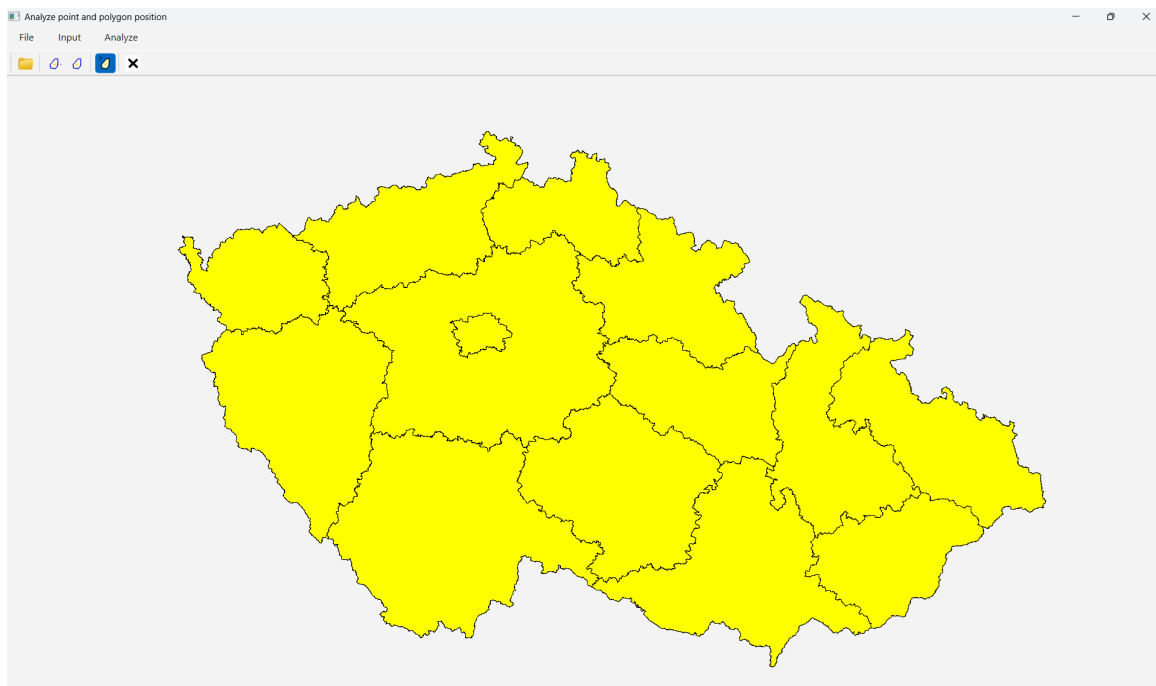
Výstupem implementované aplikace není datový soubor v klasickém smyslu, ale především vizuální a textová interpretace výsledku analýzy. Cílem je jednoznačně určit polohu zadaného bodu vzhledem k polygonové mapě a tuto informaci přehledně prezentovat. Základním výstupem je klasifikace testovaného bodu q vůči jednotlivým polygonům. V závislosti na výsledku algoritmu, jak již bylo, řečeno, může být bod vyhodnocen jako:

- ležící uvnitř polygonu
- ležící vně polygonů
- ležící na hraně polygonu
- totožný s vrcholem polygonu

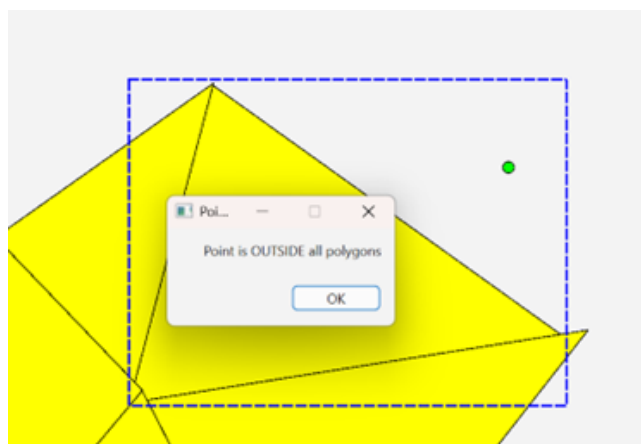
Tato informace je sdělena prostřednictvím dialogového okna, které obsahuje slovní popis výsledku. V případě, že bod leží na hraně nebo ve vrcholu sdíleném více polygony, aplikace tuto skutečnost explicitně uvádí a informuje o počtu dotčených objektů. Vedle textového výstupu hraje roli také grafická reprezentace výsledku. Polygon (nebo více polygonů), který odpovídá výsledku analýzy, je zvýrazněn přímo v mapovém okně. Zvýraznění je realizováno změnou barvy výplně, čímž je dosaženo okamžité vizuální identifikace příslušného objektu. Součástí výstupu je rovněž vizualizace optimalizačního kroku pomocí min-max boxů. Pro všechny kandidátní polygony, které prošly rychlým testem obalového obdélníku, jsou tyto boxy vykresleny přerušovanou čarou. Uživatel tak získává i přehled o průběhu výpočtu a redukci vyhledávacího prostoru. Z hlediska formátu lze tedy výstup charakterizovat jako interaktivní vizualizaci kombinující grafickou a textovou složku.

Kapitola 8

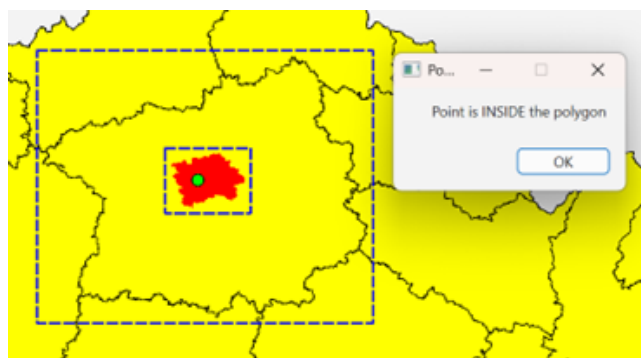
Vizualizace vytvořené aplikace



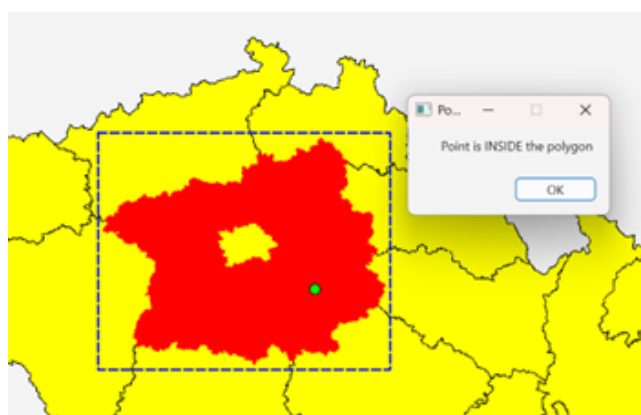
Obrázek 8.1: Načtení vstupních dat (krajů ČR)



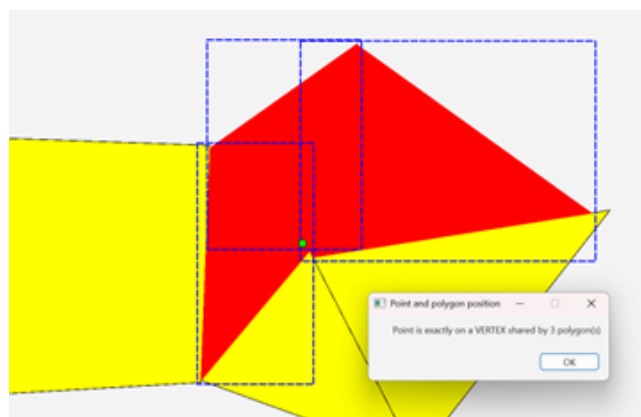
Obrázek 8.2: Výstup pro situaci, kdy je bod mimo polygon



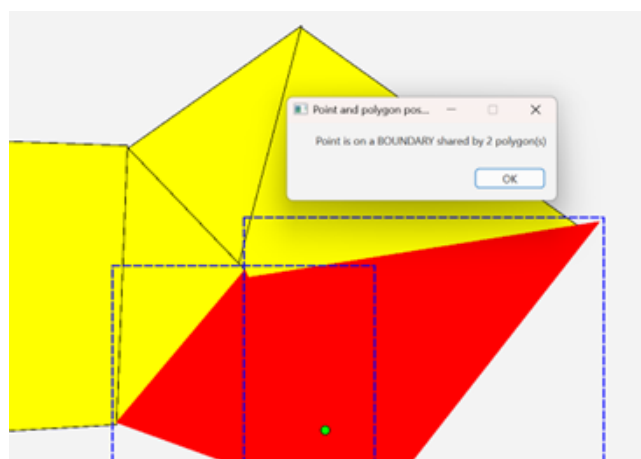
Obrázek 8.3: Bod je uvnitř polygonu



Obrázek 8.4: Situace pro polygon s dírou



Obrázek 8.5: Bod je na vrcholu, jež sdílí více polygonů



Obrázek 8.6: Bod je na hraně dvou polygonů

Kapitola 9

Dokumentace tříd, popis datových položek

Implementace aplikace je rozdělena do tří hlavních skriptů: `algorithms.py`, `draw.py` a `MainForm.py`. Rozdělení odpovídá logickému oddělení výpočetní části, prezentační vrstvy a řídicí logiky aplikace.

Skript `algorithms.py`

Skript `algorithms.py` obsahuje třídu `Algorithms`, která implementuje výpočetní operace související s určením polohy bodu vůči polygonu. Třída obsahuje hlavní položku tolerance která slouží k ošetření numerických nepřesností při výpočtech.

`createMinMaxBox(poly_rings)`

Metoda slouží k výpočtu obalového obdélníku (bounding boxu) pro daný polygon. Prochází všechny vrcholy všech prstenců polygonu a určuje minimální a maximální souřadnice v osách x a y . Tento box je využíván jako optimalizační krok pro rychlé vyloučení polygonů, které nemohou obsahovat testovaný bod.

`getPointPolygonPositionRC(q, poly_rings)`

Implementace algoritmu Ray Crossing. Princip spočívá v počítání průsečíků paprsku vedeného

z testovaného bodu s hranami polygonu. Nejprve se kontroluje, zda bod neleží ve vrcholu polygonu. Následně se testuje, zda bod neleží na některé hraně polygonu. Pokud bod neleží ani na hraně, ani ve vrcholu, provede se samotný Ray Crossing test. Výstupem metody je: 1 – bod leží uvnitř polygonu, 0 – bod leží vně polygonu, -1 – bod leží na hraně polygonu, -2 – bod splývá s vrcholem polygonu

getPointPolygonPositionWinding(q, poly_rings)

Implementace algoritmu Winding Number. Tento algoritmus vyhodnocuje celkový úhel, který polygon „opíše“ kolem testovaného bodu. Nejprve se zase ošetřují singulární případy (vrchol, hrana). Následně se pro každou dvojici sousedních vrcholů vypočítá orientovaný úhel. Úhly se sčítají s ohledem na orientaci a pokud absolutní hodnota výsledného úhlu odpovídá hodnotě 2π , bod se nachází uvnitř polygonu.

Skript draw.py

Skript draw.py obsahuje třídu Draw, která zajišťuje vykreslování polygonů, testovaného bodu a výsledků analýzy. `__pols` je seznam polygonů, kde každý polygon může obsahovat více prstenců (vnější hranice a díry), `__q` je testovaný bod. `Highlighted_polygons` je seznam polygonů, které mají být graficky zvýrazněny a `minmax_boxes` je seznam obalových obdélníků pro vizualizaci optimalizace.

mousePressEvent(e) zpracovává kliknutí myši.

paintEvent(e)

Hlavní vykreslovací metoda. Postupně vykresluje všechny polygony (žlutá výplň), zvýrazněné polygony (červená), min-max boxy (modrá přerušovaná čára) a testovaný bod (zelený). Pro vykreslení polygonů s dírami je použit QPainterPath s pravidlem Odd-Even Fill.

drawPolygons(polygons)

Převádí vstupní data (NumPy pole) na objekty QPolygonF.

setHighlightedPolygons(pols) a *setMinMaxBoxes(boxes)*

Slouží k aktualizaci grafické reprezentace výsledků analýzy.

clearData()

Vymaže všechna data a resetuje stav aplikace.

Skript MainForm.py

Skript MainForm.py obsahuje třídu Ui_MainForm. Třída Ui_MainForm představuje řídicí vrstvu aplikace. Zajišťuje komunikaci mezi uživatelem, výpočetní částí a vykreslovací komponentou. Obsahuje logiku načítání dat, spuštění analýzy i prezentaci výsledků. Datové položky jsou Canvas, která slouží k instanci třídy Draw, (sloužící jako hlavní vykreslovací plocha) a GUI komponenty – menu, toolbar a akce pro ovládání aplikace.

getPolygonsFromSHP(file_path)

Implementuje čtení binárního formátu Shapefile bez použití externích knihoven. Načítá jednotlivé části polygonu (rings) a ukládá je do vnitřní struktury.

normalizeData()

Transformuje souřadnice polygonů tak, aby se vešly do zobrazovacího okna. Provádí škálování a centrování dat.

analyzePointAndPositionClick(method)

Hlavní metoda aplikace pro analýzu polohy bodu. Nejdříve načte bod a seznam polygonů. Pro každý polygon vytvoří min-max box, provede rychlý test, a aplikuje zvolený algoritmus (Ray Crossing nebo Winding Number). Dále vyhodnotí výsledky a také zajistí grafické zvýraznění a zobrazí výsledků v dialogovém okně.

analyzeRC() a *analyzeWN()*

Jedná se o obslužné metody pro spuštění konkrétního algoritmu.

changeStatusClick() a *clearClick()*

Zajišťují interakci s uživatelem – přepínání režimu a reset aplikace.

Kapitola 10

Závěr

V rámci této práce jsme implementovaly řešení úlohy lokalizace bodu v polygonové mapě (Point-in-Polygon Problem) s využitím algoritmů Ray Crossing a Winding Number. Součástí řešení bylo také ošetření singulárních případů, práce s polygonovými daty ve formátu Shapefile a implementace jednoduchého grafického rozhraní v prostředí PyQt. Navržené řešení umožňuje korektně určovat polohu bodu i pro komplexnější geometrie, včetně polygonů s dírami, a poskytuje okamžitou vizuální zpětnou vazbu. Implementace min-max boxu zároveň přispívá ke zrychlení výpočtu tím, že omezuje množinu analyzovaných polygonů. Přestože aplikace splňuje požadavky zadání, lze identifikovat několik oblastí, které by bylo možné dále rozšířit či vylepšit.

Jedním z možných vylepšení je úprava uživatelského rozhraní tak, aby při změně polohy testovaného bodu došlo automaticky k vymazání předchozího výsledku. V současné implementaci zůstává zvýraznění předchozího polygonu zachováno, což může být pro uživatele matoucí. Dalším krokem by mohlo být plně automatické spuštění analýzy bez nutnosti manuálního výběru algoritmu. Analýza by se tak prováděla ihned po změně polohy bodu, což by vedlo k plynulejšímu a intuitivnějšímu ovládání aplikace.

Z algoritmického hlediska by bylo přínosné provést detailnější porovnání implementovaných metod Ray Crossing a Winding Number z hlediska výpočetní náročnosti, přesnosti a stability výsledků. Zajímavým rozšířením by bylo také grafické znázornění průběhu algoritmů, například vykreslení paprsku u metody Ray Crossing nebo vizualizace jednotlivých úhlů u Winding Number, což by přispělo k lepšímu pochopení jejich principu.

Z hlediska vstupních dat by bylo přínosné rozšířit podporu o další formáty (např. GeoJSON) a umožnit současné načtení více vrstev nad sebou. Tím by bylo možné analyzovat složitější

prostorové vztahy mezi objekty z různých datových sad. Další potenciální rozšíření spočívá v implementaci pokročilejších prostorových struktur (např. R-tree), které by umožnily efektivnější vyhledávání kandidátních polygonů a tím i zrychlení výpočtu u rozsáhlých datových souborů.

Kapitola 11

Seznam literatury

BAYER, T. (2026): Algoritmy počítačové kartografie, Point location problem, 2. přednáška, Přírodovědecká fakulta, Karlova Univerzita.

RIVERBANK COMPUTING (2026): PyQt6 Documentation, dostupné z:
<https://www.riverbankcomputing.com/static/Docs/PyQt6/index.html>